

Paper RAG Agent 项目技术手册

副标题：面向课程交付、简历项目和面试答辩的 Agentic RAG 工程实战

版本：Beta 项目手册 项目路径：paper-rag-agent 核心入口：/workspace/paper-rag 适用对象：想把 Agent/RAG 项目写进简历，并能讲清楚工程细节的学生

1. 这不是一个普通 Demo

很多简历里的 Agent 项目停留在“调用一个大模型，再接一个向量库”。这种项目很容易被追问击穿：为什么要用 Agent？为什么这样切 chunk？为什么检索结果可信？如何避免模型胡编引用？如何验证效果？如果依赖挂了怎么办？

本项目的定位更接近一个可运行的“学术论文研读系统”：

- 可以 ingest 论文，解析 PDF，切 chunk，写入 SQLite 和 Qdrant。
- 可以用混合检索回答论文问题，并返回 chunk 级引用。
- 可以做 no-evidence 拒答，避免对无关问题硬编答案。
- 可以接 DeerFlow 的工作台 UI，形成一个完整产品闭环。
- 可以记录反馈，把坏例子收集成 hard cases，再进入 golden set 优化。
- 可以用 smoke test 和 golden eval 验证集成和 RAG 效果。

这就是它适合作为课程项目和简历项目的原因：它不是只展示“我会调 API”，而是展示“我理解 LLM 应用从数据、检索、生成、产品到验证的完整链路”。

2. 一句话介绍

一句话版本：

Paper RAG Agent 是一个集成在 DeerFlow 工作台里的 Agentic RAG 学术论文问答系统，支持论文发现、论文入库、混合检索、证据驱动问答、引用校验、拒答机制、反馈闭环和三层 RAG Eval 回归评测。

三十秒版本：

这个项目解决的是“让大模型可靠发现、阅读和回答论文问题”。后端用 Python 实现 topic discovery、论文解析、chunk 构建、BGE-M3 embedding、SQLite/FTS5 和 Qdrant 混合检索，再通过 query rewrite、HyDE、rerank、reflect 和 abstain 组成 Agentic RAG 主路径。前端嵌入 DeerFlow 的 Next.js 工作台，提供 Discovery Loop、QA、Knowledge Builder、Wiki、Ingest、Feedback、Inbox 和 Subscription 工作流。项目还提供 smoke test、secret scan、retrieval/citation/claim 三层 eval 和 LLM-assisted recall 对比，用来证明效果不是只靠肉眼演示。

简历版本：

构建了一个面向学术论文的 Agentic RAG 系统，并集成到 DeerFlow 工作台中。系统采用 BM25/FTS5 + Qdrant dense retrieval + RRF fusion + BGE rerank 的混合检索架构，引入 query rewrite、HyDE、反思式多轮检索、三档 abstain 拒答、chunk 级引用校验和反馈驱动的 hard-case 数据闭环。实现了 FastAPI 网关、Next.js 工作台 UI、retrieval/citation/claim 三层评测和本地 smoke gate，支持真实论文问答、Wiki 生成、论文入库和订阅管理。

3. 技术栈总览

层级	技术	在项目中的作用
前端	Next.js, React, TypeScript	DeerFlow workspace UI，承载 Paper RAG 产品闭环
网关	FastAPI, Pydantic, SSE	对外暴露 /api/paper_rag/* 接口，支持同步 QA 和流式 QA
Agent 运行时	DeerFlow, LangGraph/LangChain 风格中间件	管理工具、线程状态、上传文件、子任务、记忆、总结和安全边界
Discovery	arXiv/Semantic Scholar adapters, candidate ranker	从研究主题发现候选论文，给出 selected/skipped reasons
RAG 核心	Python package paper_rag	论文发现、检索、问答、拒答、引用校验、反馈、Wiki、主动订阅
向量检索	Qdrant embedded/server	存储 chunk embedding，进行语义召回
稀疏检索	SQLite FTS5 / rank-bm25	关键词召回，弥补 dense retrieval 对精确术语的弱点
Embedding	BAAI/bge-m3 via FlagEmbedding	将 chunk 和 query 映射到语义向量空间
Rerank	BAAI/bge-reranker-v2-m3	对候选 chunk 做 cross-encoder 相关性重排
LLM	OpenAI-compatible API	DeepSeek/OpenAI/Qwen 等兼容协议模型生成答案与 rewrite
PDF 解析	PyMuPDF fallback, optional MinerU	从论文 PDF 中抽取文本、结构和多模态线索
评测	JSONL golden set, pytest, smoke scripts	评估 retrieval、citation、no-answer 和产品 API
工程安全	.env gitignore, secret scan	避免把 API key 和 runtime data 提交进仓库

3.1 为什么这些技术栈组合在一起

这个项目的价值不在于“每个组件都很新”，而在于组件之间的职责边界比较清楚。课程里要反复强调：一个可交付的 LLM 项目不是堆模型，而是把数据、模型、检索、产品和验证组合成稳定链路。

选择	替代方案	本项目选择它的原因
FastAPI	Flask, Django	类型清晰、Pydantic model 友好、适合 API adapter 和 SSE
SQLite	Postgres, MongoDB	本地开箱即用，适合课程项目和单机 demo，便于学生理解 schema
Qdrant embedded	FAISS, Chroma, Qdrant server	既能本地无 Docker 运行，也能迁移到 server 模式
FTS5/BM25	只用 dense retrieval	保留关键词和术语召回能力，适合论文场景
BGE-M3	OpenAI embedding, Jina embedding	本地 embedding 可控，支持长文本，课程里能讲清楚向量化过程
OpenAI-compatible API	绑定单一供应商	可以接 DeepSeek、OpenAI、Qwen 等模型，降低供应商锁定
DeerFlow workspace	自写简单 Chat UI	有 Agent runtime、中间件、工具、线程和工作台产品形态

3.2 学生需要理解的“工程取舍”

1. 本地 demo 优先：用 embedded Qdrant 和 SQLite，降低启动门槛。
2. 可迁移：配置里保留 Qdrant server URL，后续可以切生产形态。
3. 可降级：reranker、LLM、dense retrieval 出问题时，系统尽量返回 evidence-only 或 BM25 fallback，而不是直接崩。
4. 可验证：所有 RAG 优化都应该经过 golden set，而不是只看一次 demo。
5. 可讲述：每个模块都有明确问题、方案、权衡和失败模式，适合面试表达。

4. 系统架构

高层链路可以概括为：

```
User
-> DeerFlow Next.js UI (/workspace/paper-rag)
-> DeerFlow FastAPI Gateway
-> paper_rag router (/api/paper_rag/*)
-> Agentic RAG pipeline
-> SQLite + Qdrant + LLM
-> answer + citations + trace + feedback
```

项目里最重要的工程边界有四个：

1. integrations/deer-flow 是宿主应用。它提供工作台、网关、认证、中间件、线程和工具运行时。
2. src/paper_rag 是可独立运行的 RAG package。它不强依赖 DeerFlow，所以可以 CLI 使用，也可以嵌入网关。
3. integrations/deer-flow/backend/packages/harness 是 DeerFlow Harness。它负责把工具、subagent、memory、middleware、sandbox 和 tracing 组织成可运行的 Agent runtime。
4. tests/eval 和 scripts 是验证层。它们让项目从“能跑”变成“可证明地能跑”。

这种拆分对课程很重要：学生不仅学会“实现功能”，还会学会“如何设计模块边界”。在面试里，这比只说用了什么模型更有说服力。

4.1 模块边界怎么讲

可以用三句话讲清楚：

- DeerFlow 负责“Agent 应用运行时”：UI、网关、线程、工具、中间件和工作台。
- paper_rag 负责“论文 RAG 业务能力”：发现、解析、入库、检索、问答、拒答、Wiki、反馈和评测。
- Harness adapter 负责“把业务能力变成 Agent 工具”：把 paper_qa、paper_search、wiki_lookup 等包装成 LangChain tools，并提供 paper-research subagent。
- scripts/tests 负责“工程可验证性”：初始化、smoke、secret scan、golden eval 和 hard case 收集。

这个边界设计的好处是：如果以后换 UI，paper_rag 仍然能作为 Python package 跑；如果以后换 RAG 核心，DeerFlow 工作台也不用重写。

4.2 请求路径和数据路径不同

很多学生会混淆 request flow 和 data flow。课程里建议分开讲：

请求路径：

```
browser -> Next.js route/rewrite -> FastAPI router -> paper_rag.answer() -> LLM/retriever ->
JSON response
```

数据路径：

```
PDF/arXiv -> parser -> chunks -> embeddings -> Qdrant/SQLite -> retrieval -> evidence context
```

请求路径决定“用户如何访问系统”，数据路径决定“系统是否能答对问题”。一个项目要经得住追问，两条路径都要讲清楚。

5. 数据入库链路

RAG 的质量首先取决于数据。项目的数据链路是：

```
paper source
-> fetch PDF / local PDF
-> parse
-> section split
```

- > chunk build
- > embedding
- > SQLite metadata
- > Qdrant vector collection
- > BM25/FTS5 sparse index

关键知识点：

- PDF 不是纯文本文件，直接 read() 往往会丢失标题、段落、公式和图表上下文。
- chunk 不能只按固定字符数切。项目通过 section/context prefix 保留标题和章节信息，避免 chunk 脱离论文上下文。
- embedding 只解决语义相似，不解决所有精确匹配问题，所以还需要 BM25/FTS5。
- SQLite 保存可解释 metadata，Qdrant 保存向量，两者职责不同。

5.1 Chunk 设计细节

项目配置里本地 chunk 设置是：

参数	当前值	含义
target_tokens	500	每个文本 chunk 的目标长度
overlap_tokens	50	相邻 chunk 保留重叠，避免答案跨边界丢失
context_prefix	title + section	给 embedding 加论文标题和章节上下文
max_length	8192	BGE-M3 embedding 最大输入长度

为什么要有 overlap：

- 没有 overlap 时，一个定义或实验结论可能刚好被切成两段。
- overlap 能提高召回完整语义单元的概率。
- overlap 太大则会增加重复 chunk、索引体积和 rerank 成本。

为什么要加 context prefix：

- 单个 chunk 可能只写 “we propose...” 或 “our method...”，脱离标题后语义不完整。
- 加上 [Title] [Section] 后，embedding 更容易保留论文身份和章节语义。
- 这对跨论文对比问题尤其重要。

5.2 Parser 的现实问题

论文 PDF 解析有三个常见坑：

1. 版面顺序错乱：双栏论文可能把左右栏混在一起。
2. 公式和表格丢失：普通文本抽取对公式、表格结构不友好。
3. 图片上下文缺失：图题、图注和正文引用可能分离。

项目当前策略是 PyMuPDF fallback 保证可运行，MinerU 作为可选增强。课程里可以把它讲成一个工程取舍：第一阶段优先让链路跑通，第二阶段再提升解析质量。

面试追问：

问：为什么不用一个向量数据库存所有东西？

答：向量数据库适合相似度召回，但不适合承担全部结构化状态。论文标题、arXiv ID、ingest 状态、chunk metadata、反馈事件、订阅状态更适合 SQLite。向量库负责 dense retrieval，SQLite/FTS5 负责 metadata 和

sparse retrieval，这样查询、调试、重建和降级都更清晰。

6. Retrieval：混合检索为什么必要

普通 RAG 常见做法是 “query embedding -> vector search -> prompt”。本项目做得更完整：

1. Dense retrieval：用 BGE-M3 embedding 在 Qdrant 中语义召回。
2. Sparse retrieval：用 FTS5/BM25 做关键词召回。
3. RRF fusion：用 reciprocal rank fusion 合并多个 ranked list。
4. Candidate pooling：对 rewrite 后的多个 query 合并候选。
5. Rerank：用 cross-encoder 对 query/chunk pair 重新排序。
6. Diversify by paper：避免一个论文占满所有上下文窗口。

为什么需要混合检索：

- Dense retrieval 擅长语义近义，比如 “factuality metric” 和 “FactScore”。
- Sparse retrieval 擅长术语、缩写、公式、模型名、论文名。
- RRF 不要求不同检索器分数在同一尺度上，只利用排名信息，适合融合 heterogeneous retrievers。
- Rerank 用 query-document pair 做更精细判断，通常比单独 embedding 相似度更准确。

项目落点：

- src/paper_rag/retrieve/hybrid.py 负责 dense/sparse 检索和 RRF。
- src/paper_rag/retrieve/rerank.py 负责 BGE reranker，并设计了 graceful fallback。
- src/paper_rag/retrieve/pipeline.py 负责 rewrite 后的候选池、模态 hint 和按 paper 多样化。

6.1 RRF 公式怎么讲

RRF 的核心公式是：

$$\text{score}(d) = \text{sum over rank lists of } 1 / (k + \text{rank}(d))$$

其中 rank(d) 是文档在某个检索器结果里的排名，k 是平滑参数。本项目本地配置里 rrf_k=60。

直觉：

- 一个 chunk 如果在 dense 和 sparse 里都靠前，它的 RRF 分数会更高。
- 一个 chunk 即使只在一个检索器里靠前，也不会被另一个检索器完全抹掉。
- RRF 不需要 BM25 分数和 cosine 分数在同一尺度上。

6.2 Rerank 的成本意识

Reranker 通常比 embedding retrieval 慢，因为它要对每个 query/chunk pair 做更精细的交互式判断。工程上不能对全库 rerank，只能先召回候选再 rerank。

本项目的策略：

1. dense 和 sparse 各自召回一批候选。
2. RRF 合并后保留 top_k * 3。

3. reranker 对候选重排。

4. 最终取 top_k，并按 paper 做多样化。

这体现了 retrieval pipeline 的常见思路：先用便宜模型扩大 recall，再用贵模型提高 precision。

6.3 Retrieval 的失败模式

失败模式	表现	优化方向
Query mismatch	用户问法和论文措辞不同	query rewrite, HyDE, alias heuristic
Chunk 太碎	top chunk 缺少完整上下文	调大 chunk 或增加 context prefix
Chunk 太大	检索命中但答案定位不准	调小 chunk，增加 rerank
Dense 失效	语义召回为空或很弱	检查 embedding 模型、Qdrant collection、向量维度
Sparse 失效	缩写/术语匹配不到	检查 FTS5 index 和 keyword extraction
Rerank 过慢	QA 延迟高	降低 candidate window 或关闭 reranker

7. Query Rewrite 和 HyDE

真实用户不会总用论文原文里的关键词提问。比如用户问“original RAG paper”，库里可能存的是完整标题。项目通过 rewrite 解决 query mismatch：

- LLM rewrite：生成 2-3 个 paraphrase variants。
- keyword extraction：生成 BM25 输入。
- HyDE：生成一个假想答案作为 dense query。
- heuristic fallback：当 LLM 不可用时，用本地规则处理常见论文别名和指标问题。

HyDE 的直觉是：用户问题可能很短，但“假想答案”更像论文内容。把这个假想答案用于 dense retrieval，有时比直接 embed 问题更容易召回相关 chunk。

面试追问：

问：HyDE 会不会引入幻觉？

答：HyDE 在这里不作为最终答案，也不会直接展示给用户。它只作为额外 dense query 参与召回。最终答案仍然只能使用真实 retrieved chunks，并且经过 citation validation 和 abstain 判断。因此 HyDE 的风险被限制在召回阶段，而不是生成阶段。

7.1 Rewrite 输出包含什么

项目里的 rewrite 不是只生成同义句，而是生成三类检索输入：

字段	用途
variants	给 dense retrieval 使用的改写问题
keywords	给 BM25/FTS5 使用的关键词
hyde	给 dense retrieval 使用的假想答案

为什么要分开：

- dense query 更需要语义自然。
- sparse query 更需要关键词紧凑。

- HyDE 更像文档内容，可以把 query-document matching 转成 document-document matching。

7.2 什么时候不应该过度 rewrite

Rewrite 不是越多越好。过度 rewrite 可能导致 query drift：问题被改写到另一个方向，召回看似相关但实际偏题的 chunk。课程里可以让学生观察 trace 中的 rewritten query 和 retrieved chunks，判断是否发生 drift。

可控策略：

- 限制 variants 数量。
- 保留原始问题作为第一个 dense query。
- 对 known weak spots 用 heuristic，而不是完全依赖 LLM。
- 通过 golden set 检查 rewrite 改动是否伤害已有问题。

8. Agentic RAG 到底 Agentic 在哪里

Agentic RAG 不是简单地“RAG + Agent 名字”。在这个项目里，Agentic 体现在几个内部决策步骤：

1. Intent classify：根据问题类型决定 top_k 和最大迭代轮数。
2. Query rewrite：主动改写问题和生成检索变体。
3. Retrieve loop：允许多轮 retrieve。
4. Reflect：判断当前 evidence 是否足够，必要时生成 follow-up query。
5. Abstain：在调用 LLM 前决定是否拒答或弱证据回答。
6. Citation guard：生成后清洗和校验引用。
7. Trace：把 intent、iters、abstain、trace_id 返回用于调试和评测。

与普通 RAG 的区别：

普通 RAG	本项目 Agentic RAG
单次 query embedding	query rewrite + HyDE + fallback variants
单路向量召回	dense + sparse + RRF + rerank
永远调用 LLM	no-evidence 时跳过 LLM
引用靠 prompt 约束	prompt + citation validation + suspicious stripping
只看最终答案	返回 trace、chunks、abstain 结果
无稳定评估	golden set + smoke + hard cases

项目不是把复杂性暴露给用户，而是把复杂性放在系统内部，最终给用户的是一个简洁的问答体验。

8.1 主执行流对应源码

src/paper_rag/rag/qa_agentic.py 可以拆成六个阶段：

阶段	作用	关键函数
history rewrite	多轮对话时把问题改写成自包含问题	_maybe_rewrite_with_history
cache	命中缓存时跳过完整 QA	_check_cache
intent + retrieve loop	判断问题类型并检索证据	classify, _retrieve_loop

阶段	作用	关键函数
no chunks short-circuit	没证据时避免无意义生成	_no_chunks_response
abstain	根据证据分数决定拒答/弱答/正常答	_decide_abstain
generation + citation cleanup	调 LLM 并校验引用	_build_user_prompt, validate_citations

8.2 Reflect loop 的教学重点

Reflect 不是让模型“反思人生”，而是让系统判断当前证据够不够回答问题：

```

retrieve -> evidence -> reflect
-> sufficient: answer
-> insufficient with follow_up: retrieve again
-> max_iter: stop

```

这个设计适合复杂问题，比如“比较 Self-RAG 和 FLARE 什么时候 retrieve”。第一次检索可能只召回 Self-RAG，reflect 可以提出 follow-up query 去找 FLARE 证据。

8.3 Agentic RAG 的边界

不是所有问题都需要 Agentic RAG。如果用户只问“某篇论文标题是什么”，简单 metadata 查询就够了。Agentic RAG 适合这些场景：

- 问题需要跨论文比较。
- 问题措辞和论文措辞差异大。
- 证据可能分散在多个 chunk。
- 系统需要判断是否证据不足。
- 需要返回可追踪的中间决策。

8A. Loop Engineering：让 Agentic RAG 可观察

很多 Agent 项目只展示最终答案，面试官很难判断系统到底有没有“Agentic”能力。这个项目把内部 RAG loop 产品化为 Loop Trace，让用户能看到一次 QA 里发生了什么：

```

intent classify
-> research memory context
-> query rewrite / HyDE variants
-> retrieve + rerank
-> reflect sufficiency
-> abstain decision
-> answer with citations
-> feedback / golden set

```

Loop Trace 不是为了把复杂性丢给普通用户，而是为了教学、debug 和面试表达。它能回答几个关键问题：

Trace 字段	能解释什么
intent	系统把问题当成 factual、reasoning 还是 explore
iterations	检索跑了几轮，每轮 query 是什么，召回了多少 chunks
reflect	当前 evidence 是否足够，是否生成 follow-up query
abstain	为什么回答、弱答或拒答
citations	最终答案引用了哪些 chunk
stopped_by	loop 是正常回答、无证据、缓存命中还是达到上限

面试里可以这样讲：

我没有把 Agentic RAG 做成不可解释的黑盒，而是把内部循环做成可观察 trace。这样调 RAG 不是只看最终答案，而是能定位问题出在 query rewrite、retrieval、rerank、reflect、abstain 还是 generation。

这就是所谓 Loop Engineering：不是让 Agent 无限循环，而是给循环设计明确的状态、上限、停止条件和观测点。

8B. Research Memory Compression：记忆压缩但不污染证据链

普通多轮 RAG 经常有两个极端：要么把所有历史塞进 prompt，导致上下文越来越长；要么只保留最近几轮，用户一换说法系统就失去研究连续性。本项目新增 Paper RAG 专属 Research Memory Compression，分三层处理：

层级	内容	作用
recent turns	最近几轮 question/answer/citations	支持短期 follow-up
session summary	压缩后的会话摘要	让长研究线程不断片
research memory	当前主题、已读论文、已确认结论、待验证问题、偏好	辅助 query rewrite 和 paper scope

核心边界非常重要：

Research Memory 只用于改写问题和延续研究上下文，不能作为最终答案证据。最终答案仍然必须重新检索 paper chunks，并通过 citation validation 绑定 chunk id。

为什么要这样设计？

- memory summary 可能有压缩误差，不能当真相来源。
- 论文问答需要可追溯证据，summary 没有 chunk-level provenance。
- 如果把 memory 当 evidence，会把“模型总结”变成新的幻觉入口。
- 把 memory 限定在 query context，可以获得多轮体验，同时保持证据链干净。

可以这样回答面试追问：

问：你做了记忆压缩，那记忆会不会导致幻觉？

答：会有这个风险，所以项目明确把 memory 和 evidence 分开。Research Memory 只影响 query rewrite 和研究上下文，不直接进入 final answer 的证据集合。答案阶段仍然只使用 retrieved chunks，并且 citation 必须来自本轮检索结果。也就是说，memory 解决“用户在研究什么”，RAG chunks 解决“答案凭什么成立”。

9. Abstain：为什么拒答是高级能力

RAG 项目最容易翻车的地方不是“答不出来”，而是“证据不足还硬答”。本项目的 abstain 模块在 LLM 前做三档判断：

决策	含义	系统行为
no_chunks	没有任何 chunk	返回 no evidence
no_evidence	分数低于低阈值	跳过 LLM，直接拒答
weak_evidence	证据较弱	调 LLM，但提示证据不足
confident	证据足够	正常生成答案

关键设计点：

- evidence score 优先使用高质量信号：score_rerank、score_dense、score。

- RRF/BM25 属于低质量信号时会 fail open，避免因 degraded retrieval 错杀正确答案。
- 阈值来自配置和 eval，不是写死在 prompt 里。
- 每次决策都会进入 trace，便于复盘。

面试追问：

问：为什么不让 LLM 自己判断证据是否足够？

答：LLM 可以辅助判断，但如果把拒答完全交给 LLM，系统仍会消耗 token，并且可能被 prompt 或上下文噪声影响。这里把 abstain 放在 LLM 前，用 retrieval score 做第一道结构化闸门。低证据问题直接跳过生成，既降低幻觉，也降低成本和延迟。

9.1 阈值怎么理解

本地配置中：

参数	当前值	含义
threshold_low	0.50	低于该值直接 no_evidence
threshold_high	0.58	高于该值认为 confident
min_chunks	3	用 top 3 个高质量分数求平均

为什么不是只看 top1：

- top1 偶然命中可能导致误判。
- top3 平均更能代表整体 evidence window 的质量。
- 但取太多 chunk 又会被长尾噪声拖低。

9.2 Fail open 和 fail closed

这是非常适合面试讲的工程点：

- Fail closed：证据弱时拒答，保护可靠性。
- Fail open：评分信号质量差时不贸然拒答，避免错误阻断。

项目里对高质量信号使用阈值，对低质量信号如 BM25/RRF 会标记 degraded 并偏向 fail open。因为 BM25/RRF 主要表达排名，不一定表达绝对相关性。

9.3 No-answer case 的意义

Golden set 里包含 H100 价格、区块链共识、天气这类 out-of-domain 问题。它们不是为了让系统变聪明，而是为了验证系统知道自己不知道。一个可靠 RAG 系统必须能区分“库里没有证据”和“模型常识能回答”。

10. Citation：引用不是装饰

论文 RAG 项目的可信度很大程度来自引用。项目要求模型只能使用 [chunk:<chunk_id>] 形式引用，生成后再校验：

- 不在 retrieved set 里的 chunk id 会被删除。
- [1]、[12] 这种数字引用会被标记 suspicious。
- (Author 2020) 这种 author-year 引用也会被标记 suspicious。
- suspicious citation 会从答案中清理，避免看起来像真实来源。

这点非常适合在简历里讲，因为它体现的是“可靠性工程”，不是单纯调模型。

面试追问：

问：引用校验能完全防止幻觉吗？

答：不能。它能防止模型编造不存在的 citation token，但不能证明每句话都完全被 citation 支持。下一步可以做 statement-level attribution 或 LLM-as-judge citation precision。当前项目先做 chunk existence 和 suspicious form stripping，是性价比最高的一层防护。

10.1 引用校验的三层防线

防线	位置	作用
Prompt 约束	生成前	要求只能复制 allowed citation tokens
Existence check	生成后	删除不在 retrieved set 中的 chunk id
Suspicious stripping	生成后	清理 [1] 和 (Author 2020) 等伪引用

10.2 为什么论文项目必须做 citation

论文问答和普通聊天不同：用户不是只要一个看似合理的回答，而是要知道“答案来自哪里”。如果不能追溯到 chunk，系统就无法用于科研、课程或面试展示。

可以这样讲：

RAG 的核心不是把上下文塞给模型，而是把答案和证据建立可审计关系。citation validation 是这个项目从 demo 走向可信系统的关键一步。

11. DeerFlow 集成：为什么不是自己写一个普通 Chat UI

如果只写一个单页 Chat UI，项目会像很多 demo 一样：没有运行时、没有工具边界、没有线程上下文、没有上传文件、没有中间件、没有可扩展 agent 能力。

DeerFlow 提供的是一个 Agent 工作台宿主：

- Gateway API：统一承接 /api/* 路由。
- Thread/run runtime：支持对话、运行、事件和持久状态。
- Skills/tools：支持工具和技能体系。
- Middleware chain：把通用能力放在模型调用前后。
- Frontend workspace：把 Paper RAG 做成产品页面，而不是命令行玩具。

本项目在 DeerFlow 中新增 paper_rag router，把 Python package 暴露成产品 API：

- /api/paper_rag/status
- /api/paper_rag/qa
- /api/paper_rag/qa/sync
- /api/paper_rag/papers
- /api/paper_rag/wiki/*
- /api/paper_rag/ingest

- /api/paper_rag/feedback
- /api/paper_rag/inbox
- /api/paper_rag/subscriptions

UI 页面在：

integrations/deer-flow/frontend/src/app/workspace/paper-rag/page.tsx

Router 在：

integrations/deer-flow/backend/app/gateway/routers/paper_rag.py

11.1 Paper RAG Router 的设计点

Router 里有几个很好的工程点：

设计	意义
lazy import paper_rag	让 gateway 在 RAG 依赖缺失时仍能启动，并通过 status 暴露问题
Pydantic request/response	API contract 清晰，前后端协作更稳
runtime status endpoint	不暴露 secret，但告诉用户 LLM、embedding、SQLite、Qdrant 是否可用
sync + stream QA	UI 可以先走简单 sync，也能扩展 SSE streaming
run_in_executor	避免阻塞 async FastAPI event loop
user_id injection	为后续多用户数据隔离预留边界

11.2 为什么要有 /status

课程项目最怕“启动失败不知道为什么”。/api/paper_rag/status 不是业务功能，而是 developer experience 功能。它能告诉学生：

- paper_rag 是否 importable。
- FlagEmbedding 是否安装。
- LLM 环境变量是否配置。
- SQLite 是否存在。
- Qdrant collection 是否可用。
- 当前是否只能 evidence-only fallback。

这会极大降低课程交付时的答疑成本。

11.3 DeerFlow Harness：从 RAG 服务到 Agent Runtime

Harness 可以理解为“让 Agent 真正跑起来的运行底座”。普通 framework 往往只给一些类、函数和抽象；harness 进一步把模型、工具、子 Agent、memory、middleware、sandbox、tracing 和运行状态都组织好，让业务能力可以被 Agent 稳定调用。

本项目里同时存在三种入口：

入口	面向对象	路径	作用
UI 入口	普通用户 / 课程演示	/workspace/paper-rag	完成 QA、Loop Trace、Knowledge Builder、Wiki、Feedback 等产品流程

入口	面向对象	路径	作用
API 入口	前端 / 脚本 / 自动化	/api/paper_rag/*	通过 FastAPI router 调用 paper_rag package
Harness 入口	DeerFlow lead agent / subagent	deerflow.community.paper_rag.tools	把 Paper RAG 暴露成 Agent 可调工具

这三种入口不是重复建设，而是服务不同场景：

- UI 入口让项目像完整产品，可以演示给学生和面试官。
- API 入口让前后端边界清晰，也方便 smoke test 和集成测试。
- Harness 入口让 Paper RAG 能被 DeerFlow 的普通 Agent 调用，而不是只能在专属页面里使用。

当前 harness adapter 落在：

```
integrations/deer-flow/backend/packages/harness/deerflow/community/paper_rag/
```

它暴露的工具包括：

Tool	用途
paper_discover	从 topic 发现候选论文，返回 score、rank_reason、skip_reason
paper_qa	对 indexed paper corpus 做 Agentic RAG QA
paper_search	按 query 找相关论文
paper_section	读取指定论文的指定章节
paper_compare	按维度比较多篇论文
wiki_lookup	查询论文概念 Wiki
export_bibtex	导出 BibTeX 引用

此外项目还注册了一个内置 subagent：

```
integrations/deer-flow/backend/packages/harness/deerflow/subagents/builtins/paper_research.py
```

paper-research subagent 的价值是把论文研究任务收束到专门上下文里：它知道优先使用 paper_qa，知道 no-evidence 时不能硬答，知道 citation 必须来自 retrieved chunks，也知道 memory summary 不能当最终证据。这比把所有提示词都塞进 lead agent 更清晰，也更符合公司项目里的职责边界。

面试时可以这样表达：

我没有把 RAG 逻辑直接写进 DeerFlow lead agent，而是把 paper_rag 做成独立 package，再通过 FastAPI router 提供产品 API，通过 DeerFlow Harness adapter 提供 Agent tool API。这样 UI、API 和 Agent runtime 三个入口共用同一套 RAG 业务能力，同时保持 harness 不反向依赖 app 层。

11.4 Knowledge Builder：把知识库搭建过程产品化

很多 RAG 项目只说“我把文档放进向量库”，但真正可讲的工程链路应该是：

```
paper source
-> fetch
-> parse
-> chunk
-> embed
-> index
-> wiki/concept note
-> QA retrieval
```

本项目把这个过程封装成 Knowledge Builder，而不是只放一个 papers list。每篇论文在 UI 中都能看到构建阶段：

阶段	作用	常见失败
fetch	从 arXiv/PDF URL 获取论文和 metadata	网络失败、PDF 不可访问
parse	用 MinerU 或 PyMuPDF 解析正文	双栏、公式、表格解析不稳
chunk	按 section 和 token budget 构建 chunks	chunk 太碎或上下文不足
embed	用 embedding 模型生成向量	FlagEmbedding 未安装、模型下载慢
index	写入 SQLite metadata 和 Qdrant vectors	Qdrant offline、向量数量为 0
wiki	生成概念笔记	LLM 不可用、无有效 text chunks

这让“知识库搭建”从黑盒变成可解释工作流。学生演示时不要只说“我用了向量数据库”，而要说清楚：数据是如何进入系统、如何被切块、如何被索引、如何暴露失败状态，以及为什么这些状态会影响最终 QA。

面试表达：

我把知识库搭建拆成 fetch、parse、chunk、embed、index、wiki 六个阶段，并在 DeerFlow UI 中展示状态。这样 RAG 失败时可以定位是数据源问题、解析问题、embedding 问题还是索引问题，而不是只看到“回答不好”。

11.5 Paper Discovery Loop：把“找论文”也做成可解释闭环

很多学生项目只从“已经有 PDF”开始讲，但真实研究工作往往先面对一个主题：我要读哪些论文？这个项目新增 Paper Discovery Loop，把找论文也纳入可观察流程：

```
research topic
-> arXiv / Semantic Scholar search
-> normalize metadata
-> dedup against indexed papers
-> relevance ranking
-> selected / skipped reasons
-> manual ingest
-> Knowledge Builder
-> QA with citations
```

Discovery Loop 的关键不是“自动爬越多越好”，而是受控、可解释、可回滚。每个候选论文都会保留 score、rank_reason、skip_reason 和 ingest_status。比如一个候选可能因为 keyword_overlap 高而 selected，也可能因为 already_indexed 被跳过。这样学生能讲清楚“为什么系统推荐这篇论文”，而不是只说“我调用了 arXiv API”。

它和 QA 证据链的边界非常重要：

- Discovery candidate 只是候选，不是最终答案证据。
- 候选必须经过 ingest、parse、chunk、embed、index 后，才可能被 QA retrieval 命中。
- 最终答案仍然必须来自 retrieved chunks 和 citation validation。
- paper_discover tool 只帮助 agent 找论文，不允许直接用 abstract 当最终论文证据。

面试表达：

我把项目从“问已有论文”扩展为“先发现候选论文，再构建知识库，再做证据问答”。Discovery Loop 负责候选召回、去重、排序和解释；Knowledge Builder 负责入库；Agentic RAG QA 负责基于 indexed chunks 回答。这样既有 Agent 的主动研究味道，又不会让未入库候选污染最终 citation。

12. DeerFlow 中间件知识点

DeerFlow 的中间件可以分为两层：Gateway 中间件和 Agent runtime 中间件。

Gateway 侧关心 HTTP 请求：

- AuthMiddleware：认证用户，给 request state 注入 user。
- CSRFMiddleware：保护状态变更请求。
- CORSMiddleware：控制跨域来源。
- Metrics router：暴露 Prometheus 格式指标。
- Lifespan hooks：启动 LangGraph runtime、channel service、admin migration。

Agent runtime 侧关心模型调用和工具执行：

- ThreadDataMiddleware：为每个 thread 建立隔离目录。
- UploadsMiddleware：把上传文件注入上下文。
- ViewImageMiddleware：处理视觉输入。
- MemoryMiddleware：异步抽取和注入记忆。
- SummarizationMiddleware：长上下文时摘要压缩。
- TodoMiddleware：复杂任务规划时跟踪 todo。
- TitleMiddleware：自动生成对话标题。
- ClarificationMiddleware：拦截澄清请求。
- LoopDetectionMiddleware：避免 agent 陷入循环。
- SafetyFinishReasonMiddleware：处理安全终止原因。
- SubagentLimitMiddleware：限制并发子任务。
- ToolErrorHandlingMiddleware：工具失败时降级。
- DeferredToolFilterMiddleware：延迟暴露工具，降低上下文和误调用风险。
- SkillActivationMiddleware：按需激活技能。

这和普通 Agent 的区别在于：普通 Agent 往往是一个 while loop；DeerFlow 更像一个可治理的运行时，把状态、工具、记忆、错误、上下文、安全和观测拆成可组合中间件。

12.1 中间件为什么比“写进 prompt”更可靠

很多 Agent demo 喜欢把所有规则写进 system prompt，比如“不要泄露隐私”“不要循环”“不要调用危险工具”。问题是 prompt 只是软约束，中间件是工程约束。

能力	Prompt 写法	Middleware 写法
限制工具	告诉模型不要乱用	在模型绑定前过滤工具 schema
限制循环	告诉模型不要重复	监控消息和工具调用模式，触发终止
错误处理	告诉模型工具可能失败	捕获 tool error，返回结构化错误或 fallback
长上下文	告诉模型简洁	在上下文过长时自动 summarization
用户文件	让模型自己找	在 before_agent 注入上传文件信息

12.2 重点中间件可以这样讲

ThreadDataMiddleware：

- 给每个 thread 准备隔离目录。

- 解决多轮对话文件和输出的归属问题。
- 面试关键词：state isolation, thread-scoped workspace。

UploadsMiddleware：

- 把用户上传文件转成 agent 可见上下文。
- 解决“文件已经上传但模型不知道”的问题。
- 面试关键词：context injection, file metadata。

SummarizationMiddleware：

- 长对话接近 token limit 时压缩上下文。
- 比简单截断更保留长期任务信息。
- 面试关键词：context window management。

DeferredToolFilterMiddleware：

- 不是所有工具一开始都暴露给模型。
- 可以降低 prompt/tool schema 体积，也减少误调用。
- 面试关键词：tool discoverability, context budget。

LoopDetectionMiddleware：

- 检测 agent 重复工具调用或无效循环。
- 防止成本失控和用户等待。
- 面试关键词：agent safety, runaway prevention。

ToolErrorHandlingMiddleware：

- 工具失败不应该让整个 agent 崩溃。
- 应该把错误转成模型可理解的信息，或走降级路径。
- 面试关键词：graceful degradation。

13. Paper RAG 产品闭环

项目的 P1 闭环不是只有 QA，而是完整产品面：

功能	价值
Discovery Loop	从 topic 找候选论文，并解释为什么选中或跳过
QA	直接回答论文问题
Knowledge Builder	查看论文 fetch、parse、chunk、embed、index 和 wiki 构建状态
Wiki	把论文概念沉淀成知识条目
Ingest	从 UI 触发论文入库
Feedback	收集用户对答案的质量反馈
Inbox	接收订阅或主动推荐消息
Subscriptions	管理兴趣主题，形成主动 agent

这个闭环适合课程展示，因为学生可以演示一个真实 workflow：

1. 入库一篇论文。
2. 在 Knowledge Builder 看到构建阶段和索引状态。
3. 提问并得到带 citation 的答案。
4. 生成 Wiki。
5. 点反馈。
6. 建立订阅。
7. 用 eval/hard cases 解释如何继续优化。

13.1 为什么这些功能能组成闭环

闭环的意思是：用户行为会反过来改善系统，而不是一次性问答。

Ingest -> Knowledge Builder -> QA -> Feedback -> Hard cases -> Golden set -> RAG tuning -> Better QA

Wiki 和 Subscription 则把系统从“被动问答”扩展到“知识沉淀”和“主动提醒”：

Paper -> Concept Wiki -> Later QA / review
Subscription -> Matcher -> Inbox -> User action

这是卖课时很重要的表达：学生不是做一个孤立接口，而是在做一个可演进的 AI 产品。

13.2 UI 状态也算工程能力

手册里建议学生关注四种 UI 状态：

- loading：请求进行中，避免用户重复点击。
- success：答案、citations、chunks、trace 可读。
- empty：没有 indexed papers、没有 inbox、没有 wiki 时要有解释。
- error：后端 503、LLM missing、Qdrant missing 时要能提示下一步。

这些看似前端细节，其实是项目能否交付给普通学生使用的关键。

14. 反馈闭环和 Golden Set

没有评测的 RAG 优化很容易变成玄学。项目提供三类数据：

- Golden set：稳定回归集，防止优化一个问题时破坏其他问题。
- Real set：探索性真实问题集，用于发现失败模式。
- Hard cases：从用户反馈中提取出来的问题，用于扩充评测。

当前命令：

```
make eval-golden
make eval-golden-qa
make eval-report
make eval-citation-audit
make eval-ablation
make eval-claims
make eval-claims-report
make eval-llm-recall
make hard-cases
make verify-p0
```

当前数据集：

- qa_set.golden.jsonl：60 条 strict regression，其中 45 条正例带 chunk-level retrieval/citation 标注，15 条 no-evidence。
- qa_set.real.jsonl：40 条 real/stress，用于探索失败模式和补充 hard cases。
- qa_set.claims.jsonl：40 条 claim-level eval，其中 30 条正例带 90 个 expected claims，10 条 no-evidence。
- qa_set.example.jsonl：最小 smoke 模板，用于新环境理解 schema 和验证 runner。

当前 retrieval-only baseline（不调用 chat API）：

Metric	Value
Positive paper recall@10	0.989
Positive paper MRR	0.989
Positive chunk recall@10	0.811
Chunk label coverage	1.000
FPR@10 before first evidence	0.000
Errors	0

当前 QA no-judge baseline（调用 chat API 生成答案，但不用 LLM judge 打分）：

Metric	Value
Citation existence	1.000
Citation precision	0.867
Citation paper precision	0.922
Must-contain coverage	0.933
No-answer success	1.000
No-answer direct abstain	0.933
Errors	0

当前 claim-level QA baseline（调用 chat API 生成答案，但不用 LLM judge 打分）：

Metric	Value
Claim recall	0.811
Grounded claim recall	0.722
No-answer success	1.000
Forbidden claim violations	0
Claim-labeled items	30 / 40
Errors	0

这里的 relevant_chunk_ids 和 citation_chunk_ids 是分开的：

- relevant_chunk_ids 用于检索评测，回答“系统有没有找到核心证据”。
- citation_chunk_ids 用于生成评测，回答“最终引用的 chunk 是否可以直接支撑答案”。
- 如果没有 citation label，runner 会回退到 relevant chunk label；如果答案没有任何 citation，precision 会 skipped，漏引由 recall 暴露。

生成前还会做 deterministic evidence selection：完整检索窗口继续作为 chunks 返回给 UI 和 trace，但 prompt 里的 evidence 和 allowed citation tokens 只来自更小的 evidence_chunks。这能减少背景 chunk 被模型随手引用。

当前 retrieval ablation：

Strategy	Paper recall@10	Paper MRR	Chunk recall@10	Avg latency
Dense only	0.922	0.974	0.767	198.22 ms
Sparse only	0.956	0.782	0.567	2.19 ms
Hybrid RRF	0.944	0.959	0.811	152.66 ms
Hybrid + rerank, no rewrite	0.989	0.959	0.733	156.95 ms
Hybrid + rerank + local rewrite	0.989	0.989	0.811	220.52 ms

当前 LLM-assisted recall 对比：

Strategy	Paper recall@10	Chunk recall@10	Rewrite gain	Harm rate	Avg latency
Baseline, no rewrite	0.983	0.717	0	0.000	246.08 ms
Local rewrite + HyDE	0.983	0.817	4	0.025	215.31 ms
LLM rewrite + HyDE	1.000	0.933	10	0.050	3467.58 ms

这组实验评的是“LLM-assisted rewrite/HyDE 是否提升召回”，不是让 LLM judge 评价答案。评分仍然是 paper_id/chunk_id 的确定性命中。

为什么 make eval-golden 不需要 API key：

它评的是检索系统，不评语言生成。

流程是：

question -> local rewrite heuristic -> dense/sparse/hybrid/rerank -> top-k chunk ids
然后把 top-k 的 paper_id/chunk_id 和 golden label 做精确对比。

这像搜索引擎离线评测：标准答案是“应该找到哪篇论文、哪几个 chunk”，
不是让另一个模型主观判断答案好不好。

make eval-golden-qa 才需要 API key，因为它会调用模型生成答案，然后检查 citation existence、citation precision、must-contain 和 no-answer success。make eval-citation-audit 会额外输出 docs/RAG_CITATION_AUDIT.md，逐条解释 citation 是 direct support、right-paper/wrong-chunk，还是 wrong-paper。

make eval-claims 也需要 API key，因为它要生成最终答案，但评分仍然不是 LLM judge。它用人工标注的 expected_claims 和 accept_patterns 检查答案是否覆盖关键语义点，再用 supporting_chunk_ids 检查这些 claim 是否真的被 citation grounding。make eval-llm-recall 用同一批 claim set 对比 baseline、local rewrite/HyDE 和 LLM rewrite/HyDE 的召回收益。LLM 在这里只参与 query rewrite，不参与打分。

LLM judge 只建议做手动质量报告，不放进快速 gate。

面试追问：

问：为什么不用 LLM judge？

答：LLM judge 可以作为后续增强，但第一阶段先用 no-judge 指标更稳定，比如 paper recall、citation existence、must-contain、no-answer success。这些指标可复现、成本低、不会受 judge 模型波动影响。等基础指标稳定后，再加入 LLM judge 做语义质量评估。

14.1 指标怎么读

指标	说明	为什么重要
positive paper recall@k	正样本问题是否召回相关论文	没召回就不可能答对
positive chunk recall@k	标注证据 chunk 是否进入 top-k	证明不是只找到论文，而是找到可引用证据
MRR	相关论文排名是否靠前	越靠前越容易进入 LLM context
nDCG@k	相关结果是否排在前面	比 recall 更关注排序质量
FPR@k	危险误召回是否出现在第一个正确证据之前	防止模型优先看到错误证据
citation existence	引用是否来自 selected evidence chunks	防止伪引用
citation precision/recall	引用是否命中 citation 标注证据	证明引用真的支撑答案
citation paper precision	引用是否至少来自相关论文	区分错论文和同论文错 chunk
claim recall	答案覆盖了多少 expected claims	检查答案有没有漏掉关键语义结论
grounded claim recall	覆盖 claim 且 citation 命中 supporting chunks	检查语义结论是否真的被证据支撑
missing claims	未覆盖 claim 列表	告诉学生下一步该补答案、补检索还是补标注
rewrite gain/harm	rewrite/HyDE 相对 baseline 的召回增益和伤害	证明 LLM-assisted retrieval 不是堆概念
must-contain coverage	答案是否包含关键概念	粗粒度检查答案完整性
no-answer success	无关问题是否拒答	防止系统乱答
no-answer direct abstain	是否在 LLM 前拒答	降低成本和幻觉

14.2 Golden set 怎么扩

课程里可以要求学生按这个格式扩展：

```
{
  "qid": "my01",
  "question": "What problem does this paper solve?",
  "intent": "factual",
  "relevant_paper_ids": ["arxiv:xxxx.xxxxx"],
  "must_contain": ["keyword"],
  "gold_answer": "Short reference answer."
}
```

扩展原则：

- 每篇论文至少 3 个 factual 问题。
- 每 3 篇论文至少 1 个 comparison 问题。
- 每个主题至少 1 个 no-answer 问题。
- 对容易失败的问题标注 notes，方便以后做 hard cases。

14.3 优化 RAG 的正确顺序

建议顺序：

1. 先看 retrieval 是否召回正确论文。
2. 再看 top chunks 是否包含答案。
3. 再看 rerank 是否把证据排到前面。
4. 再看 abstain 是否误拒或漏拒。
5. 最后才看 LLM answer style。

不要一上来调 prompt。很多 RAG 问题不是生成问题，而是证据没有进入上下文。

15. 和传统 Agent 项目的区别

普通 Agent 项目常见结构：

user -> prompt -> LLM -> tool call -> answer

这个项目的结构：

user
-> intent/rewrite
-> hybrid retrieval
-> rerank/diversify
-> reflect loop
-> abstain gate
-> grounded generation
-> citation validation
-> feedback/eval loop
-> product UI workflow

差异总结：

- 普通 Agent 重在“能调用工具”；本项目重在“工具调用后的证据质量和可验证性”。
- 普通 RAG 重在“召回几个 chunk”；本项目重在“召回、筛选、拒答、引用和评测”。
- 普通 demo 重在“看起来能答”；本项目重在“答错时能定位原因，改动后能回归验证”。

15.1 可以用这张对比表讲给面试官

维度	普通 Chatbot	普通 RAG	本项目
数据来源	模型参数	向量库 chunk	Discovery 候选 + 论文解析 + SQLite + Qdrant + FTS5
检索	无	单次向量检索	rewrite + dense/sparse + RRF + rerank
可靠性	靠模型自觉	靠 prompt	abstain + citation validation + eval
工程形态	一个页面	一个接口	DeerFlow workspace + FastAPI router + product workflows
调试能力	看日志	看 chunks	trace, abstain, chunks, eval JSON
优化方式	改 prompt	改 chunk/prompt	golden set + hard cases + pipeline tuning

16. 学生应该掌握的知识点地图

RAG 基础：

- Embedding 的作用和局限。
- Chunking 的目标、overlap、section context。
- Dense retrieval vs sparse retrieval。
- Hybrid retrieval 和 RRF。
- Rerank 的意义和成本。
- Citation grounding。
- Abstain/no-answer 机制。

Agentic RAG：

- Query rewrite。
- HyDE。

- Reflective retrieval。
- Multi-step internal trace。
- Loop Trace 的可观察性设计。
- Research Memory Compression 的作用和边界。
- Tool/runtime 和普通函数调用的边界。
- 什么时候需要 Agent，什么时候普通 RAG 足够。

工程能力：

- FastAPI router 设计。
- Pydantic request/response model。
- SSE streaming。
- Next.js 产品页面。
- Knowledge Builder 状态建模。
- Local config 和 .env 安全。
- SQLite/Qdrant 双存储。
- graceful fallback。
- eval 和 smoke test。

面试表达：

- 为什么这样设计？
- 遇到依赖缺失怎么降级？
- 如何证明效果？
- 如何避免 hallucination？
- 如何继续扩展？

16.1 学习路线建议

第一阶段：跑通

- 理解 README quickstart。
- 能启动 backend/frontend。
- 能 ingest 一篇论文。
- 能问答并看到 citations。

第二阶段：理解

- 画出 ingest pipeline。
- 画出 QA request flow。
- 解释 dense/sparse/RRF/rerank。
- 解释 Loop Trace、Research Memory、abstain 和 citation validation。

第三阶段：优化

- 新增 golden questions。
- 找一个失败 case。
- 修改 rewrite/chunk/rerank/abstain 中的一个点。
- 跑 eval 对比前后结果。

第四阶段：表达

- 准备 30 秒简历描述。
- 准备 3 分钟项目讲解。
- 准备 10 分钟深挖技术点。
- 准备 5 个面试追问答案。

17. 简历写法

简历里不要把这个项目写成“我做了一个 RAG 聊天机器人”。面试官看到这种描述，很容易把它归类为模板项目，然后追问两三个基础问题就结束。更好的写法是把项目拆成四层能力：业务场景、架构设计、关键算法、工程验证。

推荐的项目标题：

Paper RAG Agent - 基于 DeerFlow 的 Agentic RAG 学术论文研读系统

一句话项目定位：

面向科研论文阅读场景，构建可本地运行的 Agentic RAG 工作台，支持论文入库、混合检索、证据驱动问答、引用校验、拒答、反馈闭环和 golden se

17.1 简历 bullet 的专业写法

一个强项目 bullet 最好包含四个元素：动作、技术、结果、可验证性。

维度	弱写法	强写法
动作	做了 RAG	设计并实现论文 ingest、retrieval、generation、feedback 全链路
技术	用了向量库	采用 BM25/FTS5 + Qdrant dense retrieval + RRF + BGE rerank
结果	可以问答	支持真实论文问答、chunk 级引用、no-evidence 拒答和 Wiki 生成
验证	跑起来了	接入 smoke test、secret scan、retrieval/citation/claim 三层 eval 和 hard-case 反馈

推荐写法：

- Paper RAG Agent - Agentic RAG 学术论文问答系统
- 基于 DeerFlow + FastAPI + Next.js 构建论文研读工作台，完成 Paper Discovery Loop、QA、Loop Trace、Research Memory、Knowledge Builder、Wiki、Feedback、Inbox、Subscription 等产品闭环。
 - 设计 BM25/FTS5 + Qdrant dense retrieval + RRF fusion + BGE rerank 的混合检索链路，提升论文术语、方法名和语义问题的召回稳定性。
 - 引入 query rewrite、HyDE、reflective retrieval 和三档 abstain 决策，将固定 RAG chain 扩展为可根据证据状态调整策略的 Agentic RAG pipeline。
 - 设计 Paper RAG 专属研究记忆压缩层，支持多轮研究上下文延续，同时保证最终答案仍回到 retrieved chunks 和 citation validation。
 - 实现 chunk 级 citation validation、伪引用清理和 no-evidence 拒答，降低无证据回答、错引和 hallucination 风险。
 - 建立 JSONL golden/claim set、retrieval-only eval、QA no-judge eval、citation audit、claim-level eval、secret scan 和 smoke gate，用数据回归验证 RAG 优化效果。

如果简历空间有限，可以压缩成三条：

- 构建 DeerFlow 集成的论文 Agentic RAG 工作台，支持论文发现、论文入库、QA、Loop Trace、Knowledge Builder、Wiki、反馈和订阅管理。
- 实现 BM25/FTS5 + Qdrant + RRF + BGE rerank 的混合检索，并加入 query rewrite、HyDE、reflect 和 abstain。
- 设计 Research Memory、citation validation、no-evidence 拒答、三层 RAG Eval 和 smoke gate，提升问答可信度与可回归验证能力。

17.2 不同学生水平的简历版本

入门版，适合刚完成课程项目的学生：

完成 Paper RAG Agent 本地部署和功能演示，支持论文入库、论文问答、引用展示、Wiki 生成和反馈记录。理解 RAG 的 PDF parsing、chunk、embedding、vector search、prompt grounding 和 citation 基础流程。

进阶版，适合能解释 RAG 优化的学生：

在 Paper RAG Agent 中扩展 query rewrite、HyDE、hybrid retrieval、RRF rerank 和 abstain 策略，分析 dense/sparse retrieval 在论文术语、缩写和语义问题上的表现差异，并通过 golden set 对比优化前后 retrieval 与 no-answer 效果。

高级版，适合希望投递 AI 应用开发、Agent 工程、LLM 平台岗位的学生：

基于 DeerFlow runtime 封装 Loop-Engineered Agentic RAG 论文研读系统，设计 Paper Discovery Loop、FastAPI adapter、同步/流式 QA、Loop Trace、Research Memory、Knowledge Builder、runtime readiness、反馈闭环和评测 gate；重点解决论文候选发现、混合检索召回、证据拒答、chunk 级引用校验、记忆不污染证据链、依赖降级和本地开箱即用问题。

更偏后端工程的版本：

设计 Paper RAG 的 FastAPI 网关与 Python package 边界，封装 `/api/paper_rag/*` 路由、Pydantic schema、runtime status、同步/流式 QA 和错误降级逻辑，使 RAG 能作为独立服务接入 DeerFlow 工作台。

更偏算法/LLM 应用的版本：

围绕论文问答场景实现 Agentic RAG pipeline：query rewrite 扩展召回意图，HyDE 构造假设答案增强语义检索，RRF 融合 dense/sparse 结果，reranker 精排候选证据，通过 Loop Trace 暴露检索/反思/拒答状态，并用 abstain 与 citation validation 控制 hallucination。

更偏产品闭环的版本：

将论文 RAG 从 CLI 能力封装为 DeerFlow 工作台产品，补齐 QA、Loop Trace、Research Memory、Knowledge Builder、Wiki、Feedback、Inbox、Subscription 和 runtime status 页面状态，覆盖 loading、error、empty、success 和 retry 等关键交互。

更偏 Agent 工程设计的版本：

设计 Paper RAG 专属 Research Memory Compression 层，将 recent turns、session summary 和 research memory 分离，用于多轮研究任务的 query context；最终答案仍强制回到 retrieved chunks 和 citation validation，避免记忆摘要污染证据链。

17.3 STAR 讲述模板

面试里不要只复述简历 bullet，可以用 STAR 结构讲：

S - 场景：普通 LLM 很难可靠回答论文问题，因为答案需要来自论文证据，并且要能追溯引用。
T - 任务：我希望构建一个能入库论文、检索证据、生成答案、校验引用、拒答无关问题的 Agentic RAG 工作台。
A - 行动：我把系统拆成 DeerFlow 工作台、FastAPI adapter、paper_rag package 和 eval scripts；检索侧使用 dense/sparse hybrid + RRF + rerank，生成侧加入 rewrite、HyDE、reflect、abstain 和 citation validation，并用 Loop Trace、Research Memory 和 Knowledge Builder 让运行过程可观察。
R - 结果：系统可以通过本地 UI 完成论文 QA、Wiki、反馈和订阅管理，并用 golden set 和 smoke gate 回归验证核心链路。

17.4 简历里可以量化什么

如果学生自己补充了实验，可以在简历里加入量化指标。没有真实实验时不要编数字，可以写“通过 golden set 评估”。

指标	可以怎么量化	面试含义
Recall@k	gold chunk 是否进入 top-k	检索是否找得到证据
MRR	正确证据排名是否靠前	rerank 是否有效
Citation validity	生成引用是否都来自 retrieved chunks	是否控制伪引用
Claim recall	答案是否覆盖 expected claims	是否漏掉关键语义结论
Grounded claim recall	覆盖 claim 且 citation 命中 supporting chunks	语义结论是否被证据支撑
Rewrite gain/harm	LLM rewrite/HyDE 相对 baseline 的收益和伤害	LLM-assisted retrieval 是否真的有效
Abstain precision	无证据问题是否拒答	是否减少 hallucination

指标	可以怎么量化	面试含义
Memory compression turns	多少轮后触发摘要	是否有长会话设计
Build stage success	fetch/parse/chunk/embed/index/wiki 是否完成	知识库搭建是否可解释
Latency	p50/p95 QA 延迟	工程性能意识
Coverage	golden set 问题类型分布	测试是否只覆盖 demo

可以写成：

基于 60 条 strict golden set 和 40 条 real/stress set 建立 RAG Eval Harness，
离线评估 paper recall@10、chunk recall@10、MRR、nDCG、FPR 和 latency，
并在 QA no-judge 模式下检查 citation existence/precision、must-contain 与 no-answer success，
进一步通过 40 条 claim-level eval 检查 claim recall、grounded claim recall 和 LLM-assisted retrieval recall，
避免只凭单次演示调 prompt。

17.5 不推荐写法和原因

不推荐写法：

用 LangChain 和向量数据库做了一个论文问答机器人。

问题：

- 太泛，看不出项目难度。
- 没有体现 DeerFlow 集成和产品闭环。
- 没有体现混合检索、拒答、引用校验和评测。
- 容易把面试官引向“你是不是套模板”的方向。

更好的表达原则：

- 不说“我用了模型”，而说“我解决了什么可靠性问题”。
- 不说“我用了向量库”，而说“dense/sparse 如何互补，为什么要 fusion 和 rerank”。
- 不说“支持问答”，而说“如何保证答案来自证据，如何处理无证据问题”。

18. 面试高频追问和专业回答

这一章的目标不是背答案，而是建立面试官追问时的“回答坐标系”。所有回答都建议遵循四步：先定义问题，再说明方案，再讲权衡，最后落到项目实现。

18.1 三分钟项目介绍

可以这样开场：

这个项目是一个面向学术论文阅读的 Loop-Engineered Agentic RAG 系统。我没有只做一个简单 chat demo，而是把它做成 DeerFlow 工作台里的完整产品：用户可以入库论文、查看 Knowledge Builder、发起 QA、查看 Loop Trace、使用 Research Memory 延续研究上下文、生成 Wiki、提交反馈和管理订阅。

技术上，后端把 paper_rag 做成独立 Python package，再通过 FastAPI router 接到 DeerFlow gateway；检索侧使用 SQLite/FTS5 的 sparse retrieval 和 Qdrant 的 dense retrieval，经过 RRF fusion 和 BGE rerank 得到证据；生成侧加入 research memory context、query rewrite、HyDE、reflective retrieval、abstain 和 citation validation，避免无证据回答和伪引用。

工程上，我补了 runtime status、smoke test、secret scan 和三层 RAG Eval。这样项目不是只在一个问题上能演示，而是可以通过检索召回、引用审核、claim 覆盖证明核心链路可复现。

18.2 面试官最可能追问的主线

追问方向	面试官想判断什么	你要主动提到什么
架构	是套模板还是理解边界	DeerFlow host, FastAPI adapter, paper_rag package, eval layer
RAG	是否懂检索质量	chunk, dense/sparse, RRF, rerank, query rewrite, HyDE
Agentic	是否真有 Agent 决策	intent, rewrite, retrieve, reflect, abstain, Loop Trace
Memory	是否懂长会话上下文	recent turns, session summary, query context only, not evidence
可信度	是否控制 hallucination	grounding, citation validation, no-evidence, suspicious citation
工程	是否能交付	config, status, fallback, tests, smoke, secret scan
产品	是否只是 API	Discovery Loop, QA, Knowledge Builder, Wiki, Ingest, Feedback, Inbox, Subscription

18.3 Agentic RAG 类问题

问：你这个项目为什么叫 Agentic RAG，而不是普通 RAG？

专业回答：

普通 RAG 通常是固定链路：用户问题 -> 检索 -> 拼 prompt -> 生成。这个项目里的 Agentic 体现在链路会根据问题和证据状态做决策：先结合 Research Memory 理解研究上下文，再判断是否属于论文问答，随后做 query rewrite 和 HyDE 扩展召回，检索后根据证据强弱决定是否继续 reflect/retrieve，最后用 HyDE 决定回答还是拒答。它不是让模型自由调用工具，而是把可控的决策点工程化，并通过 Loop Trace 暴露出来。

可以补一句权衡：

我没有把所有步骤都交给一个大模型 agent 自由规划，因为论文 QA 对稳定性和可解释性要求更高，所以采用 constrained agentic workflow。

问：Agentic RAG 和传统 Agent 的区别是什么？

答：

传统 Agent 更强调通用任务规划和工具调用，比如搜索、写文件、执行代码。Agentic RAG 更聚焦知识问答质量，它的动作主要围绕检索和证据：改写、HyDE、检索、重排、反思证据、拒答、生成带引用答案。也就是说，Agentic RAG 的目标不是“能做更多事”，而是“围绕证据做更可靠的问答决策”。

问：为什么不用一个 LangChain Agent 直接完成？

答：

因为这个项目的核心不是让 Agent 自由调用工具，而是要让论文 QA 链路可控、可调试、可评测。自由 Agent 的轨迹可能每次不同，难以做 citation v golden set 回归。这里把关键步骤显式拆出来，能记录 trace，也能单独评估 retrieval、rerank、abstain 和 generation。

问：你加了 Research Memory，为什么还要每次重新检索？

答：

Research Memory 解决的是长会话里的研究连续性，比如用户正在比较哪些论文、已经确认过哪些方向、接下来想追问什么。但 memory summary 本身不是原始证据，压缩过程也可能丢细节，所以不能直接当答案来源。项目里 memory 只辅助 query rewrite 和 paper scope，最终回答仍然必须重新检索 chunks，并且 citation 必须来自本轮 retrieved set。

18.4 Retrieval 类问题

问：为什么用了 BM25/FTS5 还要用 dense retrieval？

答：

论文问题同时有精确匹配和语义匹配。BM25/FTS5 对模型名、指标名、缩写、公式符号更敏感，比如 "Self-RAG"、"RRF"、"FactScore"。Dense retrieval 更擅长语义表达，比如用户问“这篇论文如何减少幻觉”，原文可能写的是 factuality 或 grounding。两者互补，所以先分别召回，再用 RRF 融合。

问：RRF 为什么适合融合 dense 和 sparse？

答：

BM25 分数和向量 cosine 分数不是同一个尺度，直接加权会很脆弱。RRF 只看排名，不要求分数可比较。一个 chunk 如果在 dense 和 sparse 中都排得靠前，融合分数会自然变高；如果只在一个检索器里很靠前，也不会被完全丢掉。

问：为什么需要 rerank？它和 embedding retrieval 有什么区别？

答：

Embedding retrieval 是 bi-encoder，query 和 chunk 分别编码，检索快，适合大规模召回，但相关性判断比较粗。Reranker 通常是 cross-encoder，会同时看 query 和 chunk，能判断更细的语义关系，所以更适合精排。工程上不能全库 rerank，所以先 dense/sparse 召回候选，再只对候选 rerank。

问：chunk size 怎么确定？

答：

chunk size 是 recall 和 precision 的权衡。太小会丢上下文，答案可能跨 chunk；太大会导致检索命中但证据不聚焦，还会浪费上下文窗口。本项目用 tokens + overlap，并加入 title/section context prefix，让 chunk 在语义上更完整。真正上线前应该用 golden set 对不同 chunk size 做对比。

问：如果 top-k 检索到了错误 chunk，怎么办？

答：

我会分三层处理：第一层改 query rewrite 和 alias，让召回更准；第二层用 rerank 和 diversify 改候选排序；第三层在生成前判断 evidence strength，如果证据弱就 abstain，而不是硬答。调试时会看 trace：原始 query、rewrite variants、dense/sparse 命中、RRF 排名、rerank 分数和最终 citations。

18.5 可信度和 hallucination 类问题

问：怎么避免模型编引用？

答：

只靠 prompt 不够，所以项目做两层约束。第一层是在 prompt 中只允许使用 retrieved chunks 的 `[chunk:<id>]`。第二层是生成后做 citation validation：检查答案里的 chunk id 是否存在于本次检索集合，不存在就清理；对 `[1]` 或 `(Author 2020)` 这种没有 chunk 来源的引用标记 suspicious。这样能把伪引用从生成问题变成可检测的工程问题。

问：为什么要设计 abstain？

答：

论文 QA 里错误回答比拒答更危险。用户问天气、个人隐私或库里没有证据的问题时，系统应该明确说证据不足。Abstain 的价值是把“看起来流畅但没 evidence count、score、citation availability 和问题类型做 no_chunks、no_evidence、weak_evidence、confident 等状态判断。

问：拒答会不会伤害用户体验？

答：

会，所以不能简单设一个很高阈值。拒答策略要区分场景：对事实性论文问题要更严格，对总结类问题可以允许证据覆盖稍宽；同时拒答时要给用户下一论文、换成论文内问题，或展示当前检索到的弱证据。目标不是多拒答，而是在证据不足时不冒充确定。

问：如何判断回答是否 grounded？

答：

我会看三个层面：答案中的关键 claim 是否能映射到 retrieved chunk；citation 是否引用了真实 chunk id；当移除这些 chunk 后答案是否还成立。如果答案里有无法从 chunk 支持的具体数字、结论或比较，就属于 grounding 风险。

18.6 DeerFlow 集成类问题

问：DeerFlow 在这里具体提供了什么？

答：

DeerFlow 是宿主工作台和 Agent runtime，不只是一个前端壳。它提供 workspace UI、gateway route、线程状态、工具/技能体系、中间件链、上传文件、运行状态和产品页面组织。paper_rag 负责论文 RAG 能力，DeerFlow 负责把这个能力变成可交互、可演示、可扩展

问：为什么不直接做一个 Streamlit 页面？

答：

Streamlit 适合快速 demo，但这个项目目标是课程和简历项目，需要展示工程边界和产品闭环。DeerFlow/Next.js/FastAPI 的组合能体现前后端分层、adapter、runtime status、错误状态、反馈闭环和工作台集成，这些更接近真实 AI 应用开发。

问：中间件在这个项目里有什么价值？

答：

中间件把横切能力从业务逻辑中拆出来，比如认证、限流、观测、PII scrub、token usage、latency tracking、recursion guard、tool allowlist。它和 prompt 的区别是：prompt 影响模型行为，中间件约束系统行为。一个可靠 Agent 项目不能只靠 prompt，还要有运行时边界。

18.7 工程交付类问题

问：这个项目如何证明不是只适配一个 demo？

答：

项目有多层验证：单元/聚焦测试检查关键函数，import smoke 检查依赖可导入，secret scan 防止泄露 key，retrieval/citation/claim eval 检查检索、引用和答案语义覆盖，runtime status 检查本地依赖是否 ready。演示时我会同时展示 UI 和 eval，而不是只问一个提前准备好的问题。

问：为什么项目里要有 runtime status？

答：

RAG 本地项目经常失败在环境，而不是业务代码，比如 FlagEmbedding 没装、Qdrant 未初始化、API key 未配置。Status endpoint 能把这些依赖状态显式暴露出来，同时不泄露敏感信息。它提升的是可运维性、教学体验和 debug 效率。

问：如果 Qdrant 或 embedding 失败怎么办？

答：

理想策略是 graceful degradation。Qdrant 失败时 dense retrieval 不可用，但 sparse retrieval 仍可工作；embedding 模型缺失时 runtime status 应明确提示安装依赖，而不是让 QA 请求无声失败。对课程项目来说，明确失败原因比“看起来卡住”重要。

问：如何设计 golden set？

答：

golden set 不能只放简单问题，至少要覆盖 factual、method、evaluation、compare、ambiguous 和 no-evidence。每条样本应该包含 question、expected paper/chunk、must_contain、must_not_contain、gold_answer、category 和 notes。正例先保证 paper-level label，再给核心样本补 chunk-level label；无证据题要包含无关领域、错误前提、超出语料和时间敏感问题。

如果要评估“答案是否完整”，还要额外做 claim-level set：把一个参考答案拆成 2-4 个 expected claims，每个 claim 写 `accept\_patterns` 和 `supporting\_chunk\_ids`。这样可以区分三类问题：检索没找到证据、引用不支撑答案、答案漏掉关键语义点。这样才能知道优

问：如何判断一次 RAG 优化有效？

答：

我会先固定 golden set 和 claim-level set，再比较优化前后的 retrieval recall、MRR、citation precision、claim recall、grounded claim recall、abstain precision、latency 和典型 hard cases。如果是 query rewrite 或 HyDE 改动，还会单独看 rewrite gain/harm，确认它是提升召回而不是把原本能命中的样本弄坏。如果一个改动只让某个 demo question 变好，但整体 recall 或拒答变差，就不能算有效优化。

18.8 项目局限和诚实表达

问：这个项目现在最大的局限是什么？

答：

它目前定位是本地 beta 和课程项目，不是生产级多租户 SaaS。SQLite 和 embedded Qdrant 适合本地开箱即用，但不是高并发部署方案；golden set 规模还可以继续扩大；PDF 解析对复杂公式、表格和双栏论文仍有误差。我的设计是先把 RAG 产品闭环、证据可靠性和评测链路做完整，再逐步扩展部

问：如果让你继续优化，你会优先做什么？

答：

我会按证据链优先级做：先扩充真实 golden set，定位失败类型；再优化 PDF parsing 和 chunk 结构；然后调 query rewrite、HyDE、rerank 和 abstain threshold；最后再做成本、缓存和线上部署。不会一上来就盲目换模型，因为很多 RAG 问题来自数据和检索。

18.9 面试避坑

不要这样答：

这个项目主要是调 prompt，让模型回答得更好。

更好的答法：

prompt 是生成控制的一部分，但这个项目更核心的是证据链：数据解析、chunk、hybrid retrieval、rerank、abstain、citation validation 和 eval。prompt 只是最后把证据组织成答案。

不要这样答：

Agent 就是能自己思考。

更好的答法：

在工程里我更愿意把 Agent 理解成带状态、工具和决策点的工作流。这个项目的 Agentic 体现在系统会根据 query 和 evidence state 决定 rewrite、retrieve、reflect、answer 或 abstain。

不要这样答：

向量数据库能解决知识库问答问题。

更好的答法：

向量数据库只解决相似召回的一部分。完整 RAG 还需要数据清洗、chunk 策略、稀疏检索、融合、重排、上下文构造、生成约束、引用校验和评测。

18.10 反问面试官

项目讲完后可以反问：

- 贵团队的 RAG 系统更关注知识库问答、客服场景，还是代码/文档检索？
- 现在评估 RAG 效果主要用人工验收、golden set，还是线上反馈数据？
- 对 Agent 系统来说，团队更重视自主规划能力，还是可控性、可观测性和评测稳定性？

这些反问能把讨论引向真实工程，而不是停留在“会不会调 API”。

19. 可演示脚本

课堂演示建议按这个顺序：

1. 打开 README，展示一键本地启动方式。
2. 启动 backend 和 frontend。
3. 打开 /workspace/paper-rag。
4. 在 Status 里确认 llm-ready; embed-ok; qdrant-ok。
5. Ingest 一篇论文或展示 Knowledge Builder 中已有 indexed papers。
6. 提问一个论文相关问题，讲 citations。
7. 提问一个无关问题，比如天气，展示 no-evidence。
8. 生成 Wiki，说明知识沉淀。
9. 点击 feedback，说明 hard cases。
10. 运行 make eval-golden，说明 retrieval 验证闭环。

11. 展示 make eval-claims-report 或 docs/RAG_CLAIM_EVAL_REPORT.md，说明答案语义覆盖和 grounded claim。

适合演示的问题：

What is Self-RAG?
How does Self-RAG use reflection tokens?
What are the retrieval evaluation metrics discussed in the indexed papers?
What is the weather tomorrow in Shanghai?

19.1 演示时的讲解词

打开 UI 时：

这个页面不是普通聊天页，它把论文 RAG 做成了工作台：左边是问题和运行状态，后面还有 Knowledge Builder、wiki、ingest、feedback、inbox 和 subscriptions。它展示的是一个产品闭环，不只是一个模型 API。

展示 citations 时：

这里的引用是 chunk 级别的，不是模型自由生成的 [1]。后端会校验 citation token 是否来自 retrieved chunks，不合法的引用会被清理。

展示 no-evidence 时：

这个问题不是答不出来，而是系统判断当前论文库没有足够证据，所以在 LLM 前就拒答。这是 RAG 系统可靠性的关键能力。

展示 eval 时：

我不是靠肉眼看一次答案来判断效果，而是分三层评测：retrieval 看 paper/chunk 是否找对，citation audit 看引用是否真实支撑答案，claim eval 看答案有没有覆盖关键语义结论。这样优化才可回归，也能解释失败到底发生在哪一层。

20. 课程作业设计

基础作业：

- 跑通本地 DeerFlow UI。
- ingest 一篇新的 arXiv 论文。
- 提出 3 个可回答问题和 1 个 no-answer 问题。
- 截图 answer、citations 和 no-evidence 状态。

进阶作业：

- 为自己的论文新增 5 条 golden set。
- 修改 query rewrite heuristic，让一个失败问题召回正确论文。
- 比较 reranker 开启/关闭后的 top-k citation 差异。
- 从 feedback 生成 hard case，并写一段失败分析。

高级作业：

- 为 citation precision 加入 statement-level 检查。
- 把 embedded Qdrant 切换成 server 模式。
- 新增一个 DeerFlow tool，把 Paper RAG QA 暴露给普通聊天 agent。
- 为 Wiki 增加跨论文 concept linking 的可视化。

20.1 评分 Rubric

评分项	分值	标准
本地运行	20	backend/frontend/status/QA 全部跑通
数据入库	15	能 ingest 新论文并解释 chunk 和 index
RAG 理解	20	能解释 dense/sparse/RRF/rerank/abstain
产品演示	15	QA、Knowledge Builder、Wiki、Feedback 至少四个流程可演示
评测意识	15	新增 golden set 并跑 eval
面试表达	15	能回答 5 个核心追问

20.2 常见扣分点

- 只会启动，不知道每个模块作用。
- 只会说“用了向量数据库”，讲不出 BM25/RRF/rerank。
- 答案没有 citation 或 citation 讲不清。
- 无关问题也让模型硬答。
- 没有 eval，只靠截图证明效果。
- 把 API key 写进代码或提交到 git。

21. 项目边界和后续路线

当前 beta 已完成：

- 真实 RAG runtime。
- DeerFlow gateway 和 workspace UI。
- QA/Knowledge Builder/Wiki/Ingest/Feedback/Inbox/Subscription 闭环。
- Golden set 和基础 RAG 优化。
- 本地安全和 smoke gate。

当前不作为课程第一阶段重点：

- 云部署。
- CI golden gate。
- 多租户权限隔离。
- 备份恢复。
- 生产监控。
- 大规模真实 golden set。
- token 成本统计。

这些不是不重要，而是课程节奏上应该后置。学生先把“可运行、可解释、可评估”的本地项目做扎实，再进入生产化。

21.1 后续可卖课扩展方向

方向一：RAG 质量提升

- 扩大 golden set 到 50-100 条。
- 为重点问题加 chunk-level ground truth。
- 加入 LLM judge，但只作为辅助指标。
- 做 query rewrite ablation。
- 做 chunk size ablation。

方向二：Agent 能力扩展

- 把 Paper RAG 封装成 DeerFlow tool。
- 让普通 DeerFlow chat agent 可以主动调用 paper QA。
- 加入多论文综述生成。
- 加入引用导出和 BibTeX。

方向三：工程生产化

- 切换 Qdrant server。
- 加入用户级数据隔离。
- 加 CI golden gate。
- 加监控和成本统计。
- 加备份恢复脚本。

22. 最终交付标准

一个学生如果想把这个项目写进简历，至少应该做到：

1. 能本地启动完整 DeerFlow UI。
2. 能 ingest 一篇新论文。
3. 能解释 chunk、embedding、Qdrant、SQLite 和 BM25 的职责。
4. 能解释 dense/sparse/RRF/rerank 的区别。
5. 能解释 Agentic RAG 主路径。
6. 能演示 no-evidence 拒答。
7. 能解释 citation validation 的必要性。
8. 能跑 golden eval，并读懂指标。
9. 能说出 2 个当前局限和 2 个后续优化方向。
10. 能把项目用 30 秒、3 分钟和 10 分钟三个版本讲清楚。

如果能做到这些，这个项目就不是“简历装饰”，而是一个真正能被追问、能被复盘、能体现工程能力的 Agent 项目。

22.1 三种讲述版本模板

30 秒版本：

这是一个集成在 DeerFlow 工作台里的 Agentic RAG 论文助手。它不是简单向量库问答，而是包含论文发现、论文入库、混合检索、query rewrite、HyDE、rerank、abstain 拒答、chunk 级引用校验、反馈闭环和 golden set 评测的完整工程项目。

3 分钟版本：

项目分四层：第一层是 DeerFlow workspace 和 FastAPI gateway，负责 UI、API、状态和中间件；第二层是 DeerFlow Harness，负责 tools、subagent、memory、middleware 和 agent runtime；第三层是 paper_rag Python package，负责 discovery、解析、chunk、embedding、SQLite/Qdrant、混合检索和 QA；第四层是 eval 和 smoke，负责验证系统是否真的可靠。RAG 主链路采用 dense + sparse + RRF + rerank，并在 LLM 前做 abstain 判断，生成后做 citation validation。这样既能发现候选论文，也能在证据不足时拒答。

10 分钟版本：

- 先讲业务目标：让大模型可靠阅读和回答论文问题。
- 再讲 Discovery Loop：topic -> candidates -> rank_reason -> manual ingest。
- 再讲数据链路：PDF/arXiv -> parse -> chunk -> embedding -> SQLite/Qdrant。
- 再讲检索链路：query rewrite -> dense/sparse -> RRF -> rerank -> diversify。
- 再讲 Agentic：intent、reflect loop、abstain、trace。
- 再讲可靠性：no-evidence、citation validation、fallback、secret scan。
- 再讲产品闭环：Discovery、QA、Knowledge Builder、Wiki、Ingest、Feedback、Inbox、Subscriptions。
- 最后讲评测：golden set、no-judge metrics、hard cases 和后续优化。

22.2 交付前自检清单

检查项	通过标准
环境	.env 本地存在，但没有提交到 git
Runtime	/api/paper_rag/status 显示 LLM、embedding、SQLite、Qdrant 可用
数据	至少有 1 篇新论文成功 ingest
Discovery	至少跑过 1 次 topic discovery，并能解释 rank_reason/skip_reason
QA	至少 3 个相关问题能返回答案和 citations
拒答	至少 1 个无关问题触发 no-evidence
Wiki	至少 1 篇论文能生成 Wiki entry
Feedback	helpful/not-helpful 能记录
Eval	make eval-golden 和 make eval-claims 能跑通
表达	能讲清楚 dense/sparse/RRF/rerank/abstain/citation

22.3 简历与面试最终检查

交付给学生前，建议让每个人用下面这张表做最后一轮自测。

检查项	合格表现
项目定位	能用一句话说明这是“论文 Agentic RAG 工作台”，不是普通聊天机器人
技术主线	能从 discovery、ingest、retrieval、generation、harness、evaluation 讲清楚链路
简历 bullet	至少包含 Paper Discovery Loop、DeerFlow Harness 集成、混合检索、拒答/引用校验、三层 RAG Eval
架构图	能手画 browser -> gateway -> harness/router -> paper_rag -> SQLite/Qdrant/LLM
RAG 深挖	能解释 dense/sparse/RRF/rerank 的职责和取舍
Agentic 深挖	能解释 query rewrite、HyDE、reflect、abstain 为什么算决策点
可靠性	能说明 citation validation 和 no-evidence 如何降低 hallucination
工程性	能说明 runtime status、fallback、secret scan、smoke test 的作用

检查项	合格表现
局限性	能诚实说出本地 beta、PDF parsing、golden set 规模等限制
后续优化	能按“先 golden set，再数据/检索，再生成，再部署”的顺序规划

最后一条建议：学生可以不追求把每个模块都讲得像论文作者一样深，但必须能讲清楚“为什么这么设计、替代方案是什么、这个设计解决了什么失败模式”。这三个问题答好了，项目就会从简历描述变成真正的工程经历。